Flarre AI Innovation Challenge 2026

Proof Before Release:

AI-Assisted Verification of Insurance Product Changes

Akuthota Pranathi

VNR Vignana Jyothi Institute of Engineering and Technology

Executive summary

A successful test report may still leave an approved insurance change inadequately checked. Proof Before Release would help a product quality-assurance reviewer connect each approved requirement to the evidence supporting it.

AI would propose testing obligations from written requirements and highlight ambiguity. Humans would approve them. Code would then check the supplied versions, scenarios, values and test outcomes, producing explained PASS, FAIL or UNKNOWN findings.

The example in plain English

A deductible is the amount a policyholder bears before applicable insurance payment. A new policy is being issued for the first time; a renewal continues an existing policy. The example raises the minimum deductible for new policies from ₹1,000 to ₹2,000 while keeping renewals unchanged.

This is a proposal for a small prototype using fictional rules and synthetic data. No application, completed experiment, customer validation or performance result is claimed.

Problem statement

Insurance companies frequently update their products by changing policy rules, deductibles, eligibility conditions, and effective dates. Before releasing these changes, teams must verify that the updated rules behave as intended and that all critical scenarios have been tested. However, a test report marked “PASS” does not necessarily prove that every requirement has been covered. Tests may use outdated configurations, miss important scenarios, or contain results that conflict with approved requirements. Such gaps can contribute to incorrect policy outcomes, billing errors, customer dissatisfaction, and costly rework. Reviewing requirements, configurations, and test reports separately can make these inconsistencies difficult and time-consuming to detect.

To address this challenge, I propose **Proof Before Release**, an AI-assisted verification agent that checks whether available testing evidence adequately supports an insurance product change. For example, if an insurer increases the deductible for new policies from ₹1,000 to ₹2,000 while keeping renewal rules unchanged, the agent will identify the scenarios that require verification and examine whether the available evidence covers them. Instead of relying only on a successful test status, it will flag missing scenarios, outdated reports, mismatched values, and contradictory results.

The solution will combine AI-powered requirement analysis with deterministic verification checks. The AI component will extract proposed testing obligations from written requirements, identify ambiguities, and cite supporting text for human review. A verification engine will then connect each approved requirement to its expected scenarios, relevant configuration, and available test results. It will classify the evidence as PASS, FAIL, or UNKNOWN, with a clear explanation and suggested next action for each finding. This traceable connection between requirements and test evidence is intended to help reviewers understand not only whether a check passed, but also why the available evidence supports—or fails to support—that conclusion.

The initial prototype will use fictional insurance rules and synthetic data, without accessing production systems or customer information. I will compare its performance with a manual or template-based review process, measuring missing-issue detection, incorrect alerts, and review effort. The intended benefits include earlier identification of testing gaps, more consistent reviews, improved traceability, and clearer explanations for reviewers. The approach could later be extended to additional insurance products and change types, subject to validation. Human reviewers will retain control over requirement approval and final release decisions.

How the solution would work

A testing obligation states what must be checked, for which policy scenario, and what the approved answer should be. Test artifacts go directly to the code-based verifier; AI helps interpret requirements, not approve results.

What each finding means

• **PASS:** supplied evidence satisfies the defined checks under the stated runner assumption. It is not a guarantee of an error-free release.

• **FAIL:** a confirmed mismatch or required scenario is unmet. **UNKNOWN:** information is missing, unsupported or ambiguous.

Each finding would show its requirement source, relevant configuration and test evidence, explanation and next action. A local review record could retain approved obligations, corrections and reasons. AI would not independently approve production releases.

Plan, evaluation and safeguards

A small, phased implementation

• **1. Create synthetic inputs:** one fictional deductible rule, approved scenarios, configuration, test results and a run manifest identifying the tested files.

• **2. Analyze requirements:** propose obligations with source citations and human approval. Add the AI stage only if it can be safely implemented and evaluated.

• **3. Verify evidence:** use code for version, policy scenario, date, value and execution-status checks.

• **4. Review and evaluate:** display a simple evidence table and run reproducible valid and deliberately defective cases.

Planned stack: Python, typed data validation, a safe test-report parser and a small FastAPI interface. No production integrations or completed components are claimed.

Evaluation: proposed measures, not results

Compare with manual/template review and an equally configured code-based checker. For AI comparisons, keep approved obligations and final checks identical. Separate development and held-out cases; freeze labels before evaluation.

Measure missing-issue detection, incorrect alerts, false-complete findings and review effort. For AI, measure citations, ambiguity, corrections and authoring time. Include valid cases so blocking everything cannot look successful.

Intended value and limits

Earlier gap detection, consistent reviews and traceable explanations are intended benefits, not proven outcomes. Existing CI/ALM and insurance testing tools may already cover the workflow; customer need and differentiation require validation. A platform feature may be the right outcome.

Use synthetic data only. Treat document text as untrusted; give AI no execution tools and require human approval. Reject unsafe XML and unknown report conventions. Hashes bind supplied content but do not prove honest runner execution. Final decisions stay with humans; no underwriting, deployment or legal interpretation is included.

Next: review a QA evidence packet with its owner and compare the existing workflow. Synthetic self-evaluation cannot establish demand or production reliability.

Technical context: GitHub rerun documentation and pytest skip/xfail documentation support version and outcome distinctions; Pact and in-toto document existing verification approaches. Sources accessed October 12, 2026.